

Report: Denoising Logistic Noise

Abhijeet Vikram

May 30, 2026

Abstract

This report presents my work on denoising a deterministic logistic noise process for image generation within a diffusion-based framework. Building on prior work, I model the reverse dynamics using a tree-based branch selection mechanism applied to a class of the CIFAR-10 dataset. I first explore a discrete branch-selection approach, and then extend it to a continuous (soft) branch-selection formulation to improve reconstruction quality. Finally, I investigate modifications to the loss function to further enhance performance. Experimental results demonstrate the effectiveness of the proposed approaches and highlight the trade-offs between discrete and continuous formulations.

1 Introduction

Diffusion models are generative models that gradually transform a data sample into noise using a fixed forward “noising” process, and then learn the reverse transformation using a neural network. The forward process is mathematically tractable and typically involves adding Gaussian noise with variance controlled by a schedule. In this report, we first summarize the classical diffusion formulation and then describe a modification where the noise is produced using the logistic map at its chaotic parameter.

2 Classical Diffusion Forward Process

The forward diffusion process is defined as a Markov chain that progressively adds noise to the data. Given a clean sample x_0 , the forward transition is:

$$x_t = \sqrt{\beta_t} x_{t-1} + \sqrt{1 - \beta_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad (1)$$

so, probability q of getting x_t given x_{t-1}

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I), \quad (2)$$

where β_t is a small variance term controlling the amount of noise injected at timestep t . By composing all t steps, we obtain a closed-form expression:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad (3)$$

where

$$\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s). \quad (4)$$

therefore,

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, \sqrt{1 - \bar{\alpha}_t} I), \quad (5)$$

This expression allows the model to train on arbitrary timesteps without explicitly simulating the entire chain.

2.1 Training Objective

The core training task is to predict the noise that was added. The model learns a function $\epsilon_\theta(x_t, t)$ and is optimized via:

$$\mathcal{L}(\theta) = \mathbb{E}_{x_0, t, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2]. \quad (6)$$

This objective is equivalent to maximizing a variational lower bound and has been shown to yield strong generative performance.

3 Reverse Process

During sampling the reverse process in diffusion models reconstructs the sample by iteratively denoising:

$$P(A | B, C) = \frac{P(B | A, C)P(A | C)}{P(B | C)} \quad (7)$$

$$q(x_{t-1} | x_t, x_0) = \frac{q(x_t | x_{t-1}, x_0)q(x_{t-1} | x_0)}{q(x_t | x_0)} \quad (8)$$

by the markovian nature of the forward process this can be written as:

$$q(x_{t-1} | x_t, x_0) = \frac{\mathcal{N}(x_t; \sqrt{\alpha_t}x_{t-1}, (1 - \alpha_t)I) \mathcal{N}(x_{t-1}; \sqrt{\bar{\alpha}_{t-1}}x_0, (1 - \bar{\alpha}_{t-1})I)}{\mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)} \quad (9)$$

$$q(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_q(x_t, x_0), \sigma_q^2(t)I), \quad (10)$$

where the mean depends on the network's noise prediction:

$$\mu_q(x_t, x_0) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \bar{\epsilon}_t \right), \quad \sigma_q^2(t) = \frac{(1 - \alpha_t)(1 - \bar{\alpha}_{t-1})}{(1 - \bar{\alpha}_t)} \quad (11)$$

here ϵ_t is the noise that drove the transition from x_0 to x_t . To deal with this, we will train a neural network to predict ϵ_t . After training, the network will be a black box which takes a value of t and a noisy image x_t as input and returns a prediction of $\bar{\epsilon}_t$ as ϵ_θ so we can get x_{t-1} by:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta \right) + \sigma_q(t)Z, \quad Z \sim \mathcal{N}(0, I) \quad (12)$$

4 Sampling or Generation

During sampling, an image of the similar shape as training images is generated completely randomly from a $Normal(0, 1)$ distribution.

$$x_t = I, \quad I \sim \mathcal{N}(0, 1) \quad (13)$$

This image is used as x_t and with t is given to the trained model which gives a prediction ϵ_θ for $\bar{\epsilon}_t$ by which we calculate x_{t-1} by using eq.(12) iteratively and reach to x_0 , the original image associated with the random image.

5 Logistic-Map-Based Noising

5.1 Motivation

Instead of injecting Gaussian random noise, one may replace it with a deterministic chaotic process. The logistic map at parameter $r = 4$ is:

$$x_{n+1} = 4x_n(1 - x_n), \quad (14)$$

is known to produce chaotic, non-repeating values in the interval $(0, 1)$ for almost all initial conditions. This when used for noising would be deterministic and hopefully easier for the model to learn than a stochastic diffusion process.

5.2 Invariant Distribution of the Logistic Map

We consider the logistic map at the fully chaotic parameter $\theta = 4$:

$$x_{t+1} = f(x_t) = 4x_t(1 - x_t), \quad x_t \in (0, 1). \quad (15)$$

We seek a probability density $\rho(x)$ that is invariant under the dynamics. That is, it satisfies the Frobenius–Perron equation:

$$\rho(x) = \sum_{y: f(y)=x} \frac{\rho(y)}{|f'(y)|}. \quad (16)$$

To find the preimages of x , we solve

$$x = 4y(1 - y). \quad (17)$$

This can be rewritten as

$$4y^2 - 4y + x = 0. \quad (18)$$

Solving this quadratic equation yields two preimages:

$$y_{1,2} = \frac{1 \pm \sqrt{1 - x}}{2}. \quad (19)$$

The derivative of the map is

$$f'(y) = 4(1 - 2y). \quad (20)$$

Evaluating at the preimages gives

$$|f'(y_1)| = |f'(y_2)| = 4\sqrt{1-x}. \quad (21)$$

We now propose the density

$$\rho(x) = \frac{1}{\pi\sqrt{x(1-x)}}. \quad (22)$$

Using the identity

$$x = 4y(1-y), \quad (23)$$

we obtain

$$y(1-y) = \frac{x}{4}, \quad \sqrt{y(1-y)} = \frac{\sqrt{x}}{2}. \quad (24)$$

Thus,

$$\rho(y_1) = \rho(y_2) = \frac{1}{\pi\sqrt{y(1-y)}} = \frac{2}{\pi\sqrt{x}}. \quad (25)$$

Substituting into the Frobenius–Perron equation:

$$\rho(x) = \frac{\rho(y_1)}{|f'(y_1)|} + \frac{\rho(y_2)}{|f'(y_2)|} \quad (26)$$

$$= \frac{1}{4\sqrt{1-x}} \left(\frac{2}{\pi\sqrt{x}} + \frac{2}{\pi\sqrt{x}} \right) \quad (27)$$

$$= \frac{1}{\pi\sqrt{x(1-x)}}. \quad (28)$$

Hence, the invariant density of the logistic map at $\theta = 4$ is

$$\rho(x) = \frac{1}{\pi\sqrt{x(1-x)}}, \quad (29)$$

which corresponds to the $\text{Beta}(\frac{1}{2}, \frac{1}{2})$ distribution.

5.3 Constructing Logistic Noise

Let i_0 denote the random image chosen from the image dataset, we begin the noising process by applying logistic eq.(14) iteratively:

$$i_t = 4i_{t-1}(1 - i_{t-1}), \quad x_n \in (0, 1) \quad (30)$$

Here i_t is known to follow the $\text{beta}(0.5, 0.5)$ PDF at $\theta = 4$ as $t \rightarrow \infty$

Now, this image i_t , with t is entered into the model and model tries to predict the choice of branch to reverse the noise.

The motivation behind using logistic noise was since, here we reach a completely noisy image $\text{beta}(0.5, 0.5)$ in almost 3 steps, compared to 10,000 steps with gaussian.

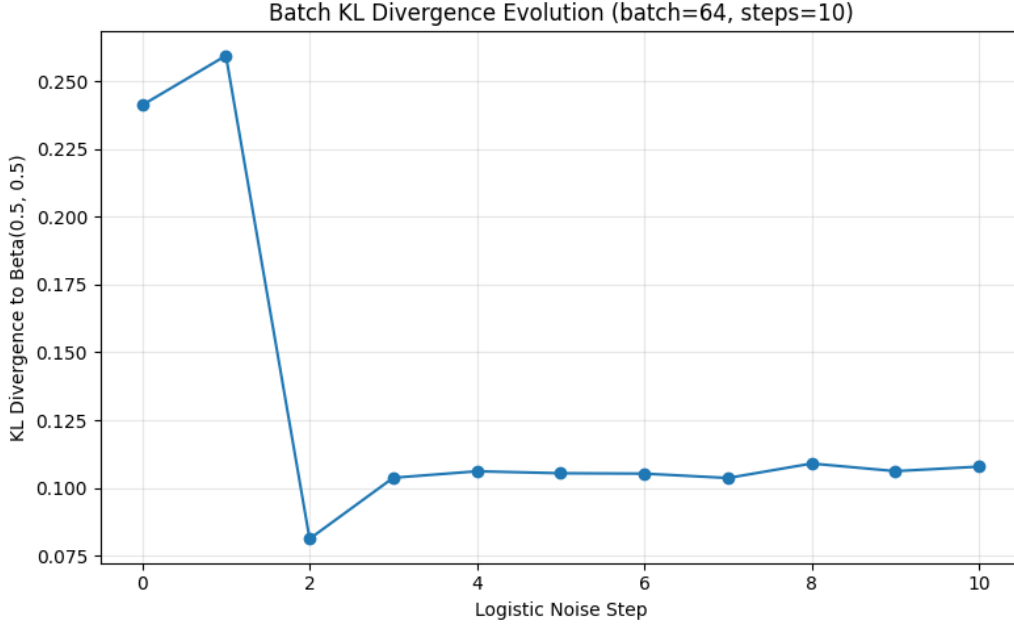


Figure 1: KL Divergence of a batch of images vs $\text{beta}(0.5, 0.5)$ across 10 time-steps. Notice the minima at second time-step, and a mostly flat curve after it indicating invariant PDF. The slight increase here is intantial to perticular batch.

5.4 Denoising Logistic Noise

Since, logistic noise is a quadratic equation it will have two solutions in reverse,

$$i_t = 0.5(1 \pm \sqrt{1 - i_{t+1}}), \quad x_n \in (0, 1) \quad (31)$$

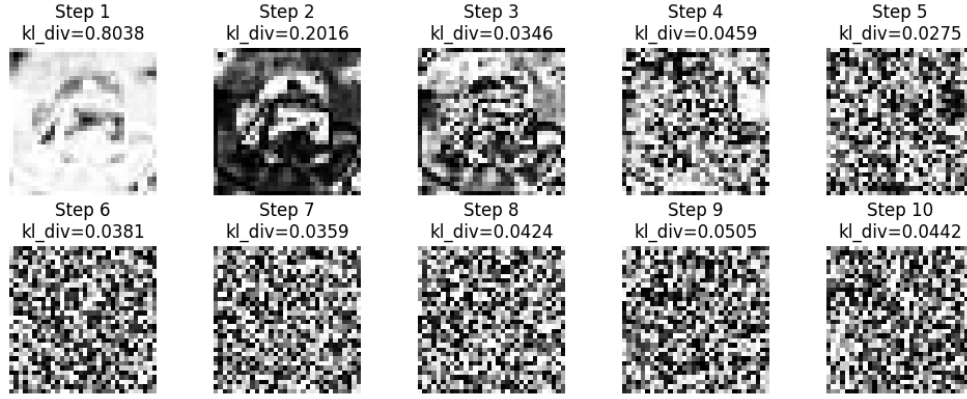


Figure 2: KL Divergence of varied across time-steps for single image from CIFAR10 dataset.

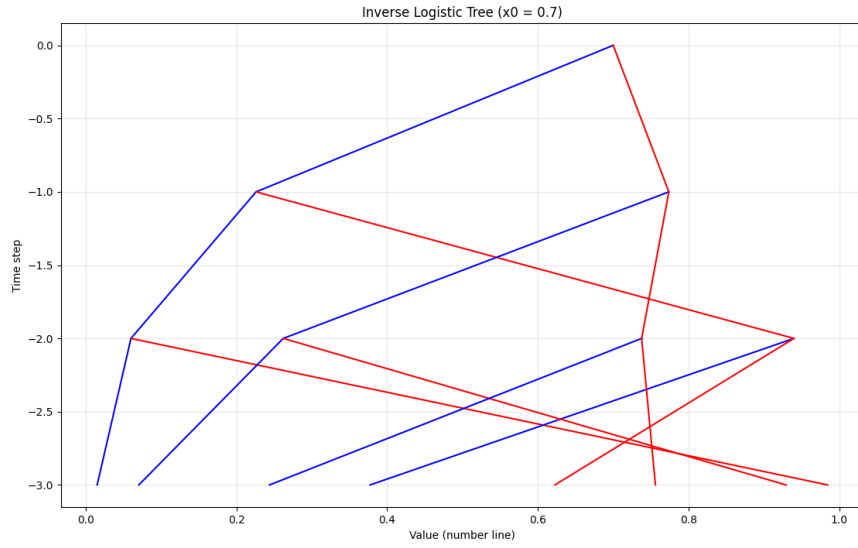


Figure 3: Branches of a pixel in reverse for $x_0 = 0.7$ and the points on number line. Model tries to choose one of these branches to undo the noise.

this forms a tree in reverse for denoising,

we train a model that takes in x_t, t and tries to pick the right branches to choose at every time-step to denoise the image back to original. The modal tries to pick the correct branch which gives the correct x_{t+1} .

5.5 Training with hard branches

Algorithm 1 Branch-Based Reverse Reconstruction Training

Require: Sequence $\{x_0, x_1, \dots, x_T\}$, model f_θ , losses $\mathcal{L}_{\text{MSE}}, \mathcal{L}_{\text{BCE}}$

Ensure: Total loss $\mathcal{L}_{\text{total}}$

```

1:  $noise \leftarrow \text{Noising}(x_t, t)$ 
2:  $\mathcal{L}_{\text{total}} \leftarrow 0$ 
3: for  $t = T, T - 1, \dots, 1$  do
4:    $x_{\text{prev}}^{\text{gt}} \leftarrow noise[t - 1]$ 
5:    $x_{\text{curr}} \leftarrow noise[t]$ 
6:    $b_{\text{gt}} \leftarrow \text{GetBranchBinary}(x_{\text{prev}}^{\text{gt}}, x_t)$ 
7:    $p \leftarrow f_\theta(x_t, t)$ 
8:    $\text{reshape } p \rightarrow (B, 1, 32, 32)$ 
9:    $\hat{b} \leftarrow \text{StraightThroughBinarize}(p)$ 
10:   $\hat{x}_{t-1} \leftarrow \text{BinaryToPrediction}(p, x_t)$ 
11:   $\mathcal{L}_{\text{MSE}} \leftarrow \|\hat{x}_{t-1} - x_{\text{prev}}^{\text{gt}}\|^2$ 
12:   $\mathcal{L}_{\text{BCE}} \leftarrow \text{BCE}(p, b_{\text{gt}})$ 
13:   $\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{total}} + \frac{\mathcal{L}_{\text{MSE}} + 20 \cdot \mathcal{L}_{\text{BCE}}}{T}$ 
14: end for
15: return  $\mathcal{L}_{\text{total}}$ 

```

5.6 Modal- DeepConv.

5.7 Sampling

z_t in the logistic equation is also shown to follow a $\text{beta}(0.5, 0.5)$ probability distribution function as shown in img.

$$z_t \sim \text{beta}(0.5, 0.5) \quad (32)$$

During sampling, a new image is randomly sampled from (32) then entered to the trained model which gives outputs a matrix used for branch selection.

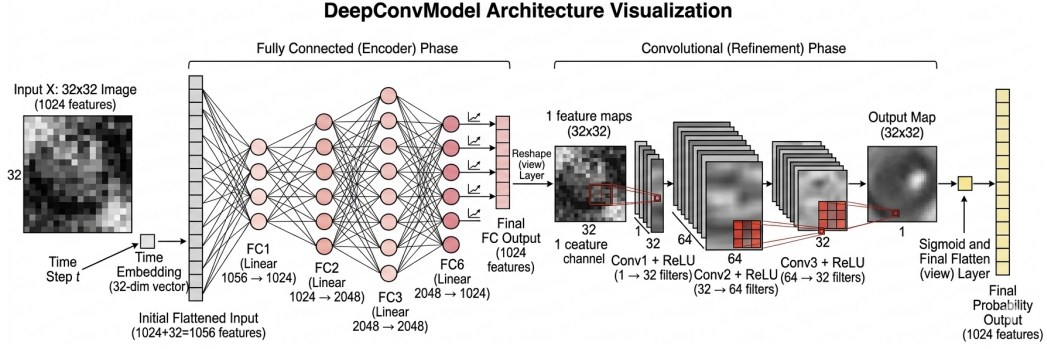


Figure 4: This includes a set of linear layers to approximate the function and a few convolution layers to tackle pixelated output.

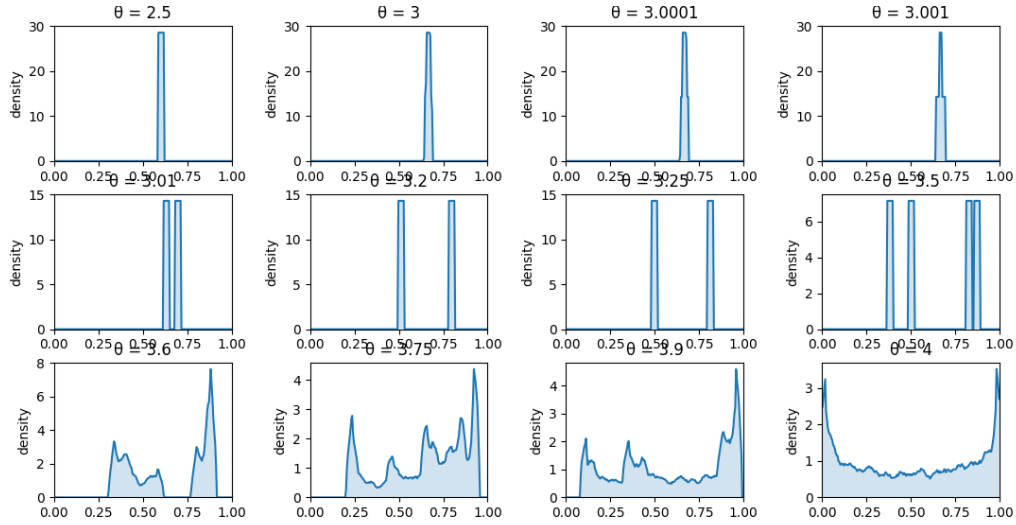


Figure 5: Probability distribution function for different x_0 for different θ . Notice U-shaped $beta(0.5, 0.5)$ distribution for $\theta = 4$, chaos.

5.8 Results

We see model learning some capabilities and edges and inlarge structure of the images but they are not very clear and highly pixelated.

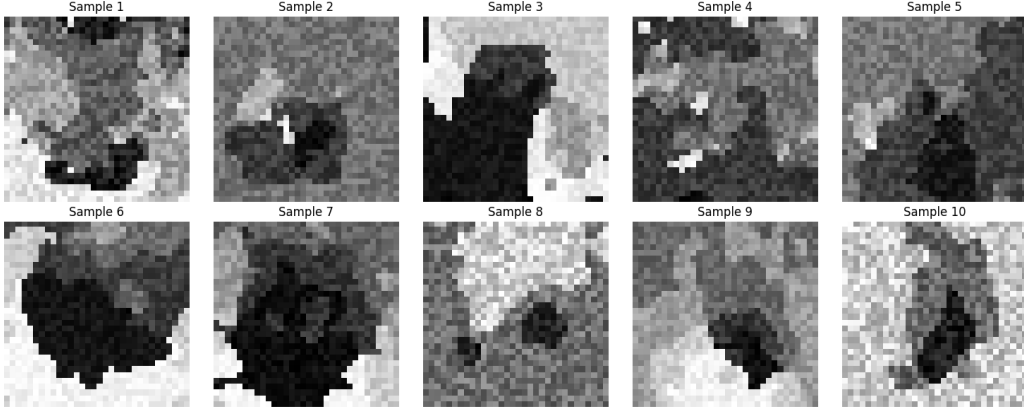


Figure 6: Results from DeepConv model trained for 100 epoch with batch size 32.

5.9 Issue

The issue with this architecture persists is, since a tree structure, once a modal commits to a wrong branch there is no way for the modal to correct itself. To tackle this we introduced soft branches which are a weighted sum of both the branches and the modal tries to predict the weights of the branch instead of two discrete branches and used MSE loss function to force towards the node and BCE loss to pick the correct branch.

$$i_t = p * 0.5(1 + \sqrt{1 - i_{t+1}}) + (1 - p) * 0.5(1 - \sqrt{1 - i_{t+1}}), \quad x_n \in (0, 1) \quad (33)$$

This tries to force the correct branch,

$$\mathcal{L}_{\text{BCE}} \leftarrow \text{BCE}(p, b_{\text{gt}}) \quad (34)$$

This tries to force the correct value of the combination for which the branches are calculated,

$$\mathcal{L}_{\text{MSE}} \leftarrow \|\hat{x}_{t-1} - x_{\text{prev}}^{\text{gt}}\|^2 \quad (35)$$

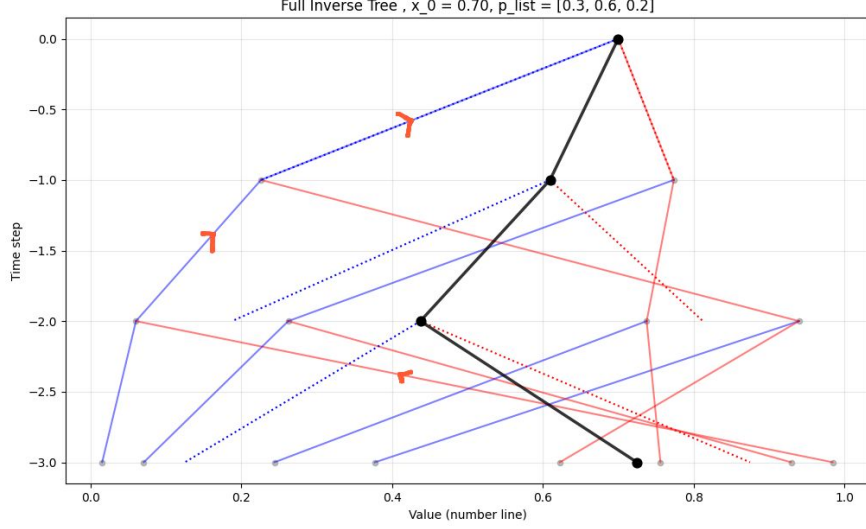


Figure 7: This graph presents the soft branches and the model predicts the values between them.

Algorithm 2 Soft Branch-Based Reverse Reconstruction Training

Require: Sequence $\{x_0, x_1, \dots, x_T\}$, model f_θ , losses $\mathcal{L}_{\text{MSE}}, \mathcal{L}_{\text{BCE}}$

Ensure: Total loss $\mathcal{L}_{\text{total}}$

- 1: $\text{noise} \leftarrow \text{Noising}(x_t, t)$
 - 2: $\mathcal{L}_{\text{total}} \leftarrow 0$
 - 3: **for** $t = T, T - 1, \dots, 1$ **do**
 - 4: $x_{\text{prev}}^{\text{gt}} \leftarrow \text{noise}[t - 1]$
 - 5: $x_{\text{curr}} \leftarrow \text{noise}[t]$
 - 6: $b_{\text{gt}} \leftarrow \text{GetBranchBinary}(x_{\text{prev}}^{\text{gt}}, x_t)$
 - 7: $p \leftarrow f_\theta(x_t, t)$
 - 8: $\text{reshape } p \rightarrow (B, 1, 32, 32)$
 - 9: $\hat{x}_{t-1} \leftarrow \text{ProbsToPrediction}(p, x_t)$
 - 10: $\mathcal{L}_{\text{MSE}} \leftarrow \|\hat{x}_{t-1} - x_{\text{prev}}^{\text{gt}}\|^2$
 - 11: $\mathcal{L}_{\text{BCE}} \leftarrow \text{BCE}(p, b_{\text{gt}})$
 - 12: $\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{total}} + \frac{\mathcal{L}_{\text{MSE}} + 20 \cdot \mathcal{L}_{\text{BCE}}}{T}$
 - 13: **end for**
 - 14: **return** $\mathcal{L}_{\text{total}}$
-

5.10 Results

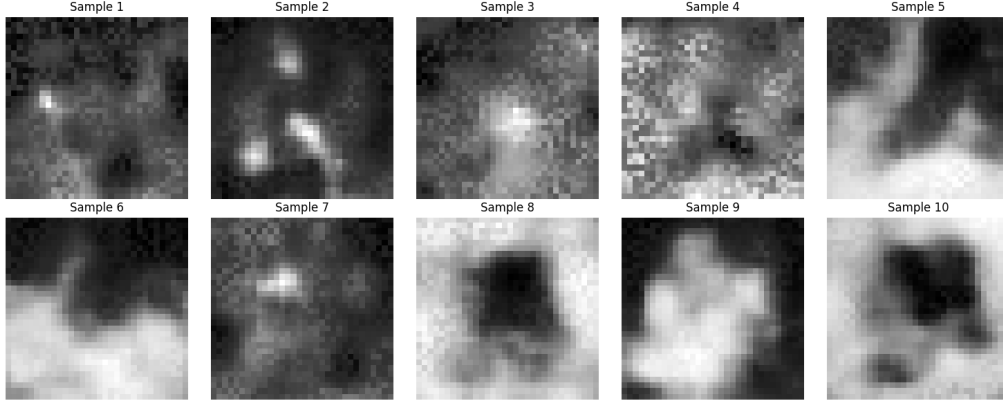


Figure 8: Results from DeepConv model trained for 100 epoch with batch size 32.

We see model learning some capabilities and edges and inlarge structure of the images but they are not very clear outputs. The modal used for this is bit smaller, to prevent overfitting,

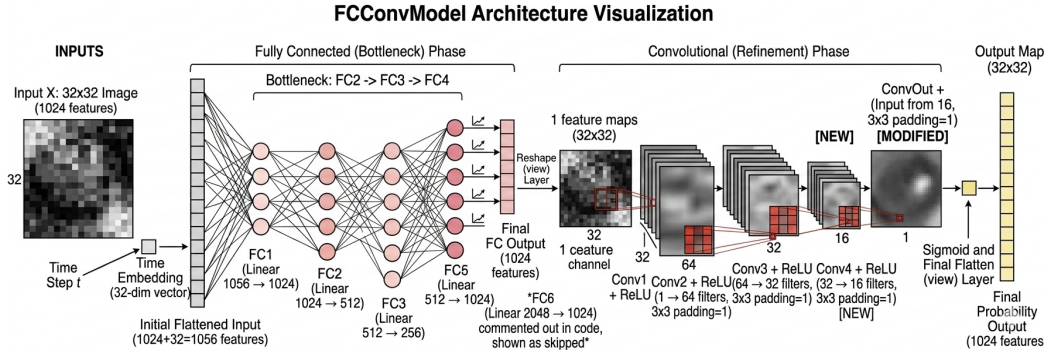


Figure 9: FCConv model - The modle used with soft branches, No. of parameters $\sim 2.5\text{M}$

6 Issue

The issue still remains with this model is that the 2 loss functions are fighting for the same things and aren't able to be optimized well after a certain threshold. The data to us was also limited (around 5000 images) I chose CIFAR10 dataset because its been heavily used in literature and in particular multiple classes would be a much difficult problem to solve so for simplicity I chose a class of CIFAR10. Also since denoising chaos itself is a very difficult task, the p values output from the modal goes very close to 0.5 suggesting modal is struggling to pick branches after certain amount of training this is leading to a plateau in the loss curve.

Another issue that I could think of is, x_t as input has no information about which branch to pick to go to x_{t-1} . In diffusion we are adding noise this meant the original signal about x_0 still remained but goes noisy here the information gets completely lost about which branch to pick. This is why I see the p values being closer to 0.5 for image pixels and few black and white pixels are easy to predict so it learns to give always black or white pixel, so we see clear outline of the subject but no clearer image. Model at this point is randomly guessing the branch, and the learning happens mostly in first epoch and rest accuracy increases very slowly 0.1 per epoch for even good learning rates.